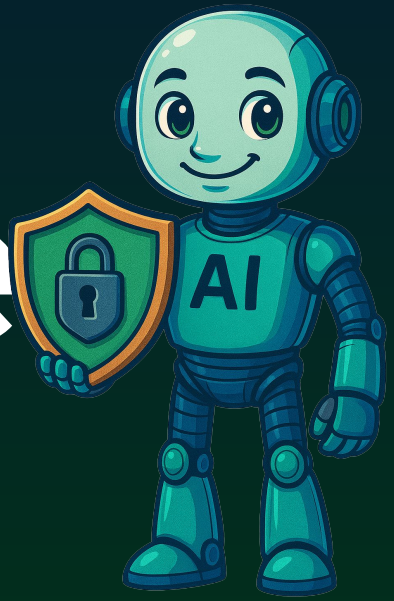
appliedAI Developers

Welcome



Program starts at 18:30

Get Updates

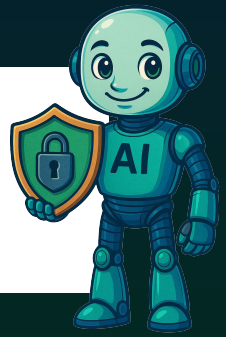
Follow us on Linkedin

appliedAI institute for europe

celonis

appliedAI Developers

Engineers, researchers, and tech leaders solving real-world production AI challenges.
We run techie-to-techie meetups to share hard learned lessons and what actually works when shipping AI at scale.



What we value?

- making AI development clear
- connecting technical stories

with real impact

Get involved:

- share your lessons learned & challenges
- network with inspiring developers

Grow with us:

- host a meetup
- share the knowledge
- expand your skill set

Follow & Contact us

appliedAI institute for europe

Disclaimer

appliedAI Developers

By attending this event today, you grant the organizers the full right to use the images resulting from photo/video recordings, as well as any reproductions or adaptations of those images, for promotional purposes. This may include (but is not limited to) the right to use them in print and online advertising, and on social media. Please contact us if you have any questions.

appliedAI institute for europe

appliedAI Developers

Agenda

18:30	Pizza Mingle & Welcome
18:45	Framing: Why state matters?
18:55	Talk #1: State & harnesses: reproducibility, checkpoints, replay
19:20	Challenge Lab
19:50	🍕 Slicebreaker
20:10	Talk #2: Memory that learns: what agents are still missing
20:35	Synthesis -> Discussion -> Networking

appliedAI institute for europe

appliedAI Developers

Why this series exists

Most AI events focus on:

- models
- tools
- demos
- benchmarks

Nexus focuses on:

- production failures
- architectural patterns
- tradeoffs
- what actually breaks after the demo works

Goal: build a production readiness blueprint for agentic systems.

appliedAI Developers

Production Readiness Blueprint

1. Context — what the system sees
2. Behavior — how it decides
3. Execution — how it acts
4. 🍕 State — what it remembers 🍕
5. Collaboration — how humans stay in control
6. Observability
7. Security
8. Learning & Adaptation
9. Scaling, Performance & ROI

appliedAI institute for europe

appliedAI Developers

What We Learned So Far

Module 1: Context

- Evaluation was the weakest link
- Fragmented data and context drift dominated
- Context quality determines downstream behavior

Module 2: Behavior

- Planning is mostly implicit
- Fallback is often undefined
- Opaque decisions create trust issues

Module 3: Execution

- Non-determinism was the dominant failure mode
- Silent failure was worse than hard failure
- State transitions were hardest to reconstruct

appliedAI Developers

State is not chat memory.

State is:

- workflow progress
- intermediate outputs
- decisions already made
- checkpoints
- memory across runs
- what allows replay, resume, and debugging

Without state, there is no reproducibility.
Without reproducibility, there is no reliability.

appliedAI Developers

Registration Signal

appliedAI
institute
for europe

appliedAI Developers

Where state management breaks most

- **Context / memory drift over time**: ~30%
- **Inconsistent state across workflow steps**: ~20%
- **Hard to resume after failure**: ~15%
- **State not traceable / debuggable**: ~15%

state quality over time: drift, inconsistency, and lack of traceability.

Current approach to state

- **Ad-hoc persistence**: ~40%
- **Workflow-level state / checkpointing**: ~30%
- **No explicit state**: ~15%
- **Fully traceable / reproducible pipelines**: ~15%

Most teams persist *something*, but few have a fully reproducible state model.

Biggest challenge managing state

- **Deciding what to persist vs discard**: ~30%
- **Keeping state consistent across steps**: ~25%
- **Scaling state across systems/services**: ~20%
- **Handling retries and partial failures**: ~20%

“The hard part is deciding what state is worth trusting.”

appliedAI Developers

Simon

State & harnesses: reproducibility, checkpoints, replay

appliedAI
institute
for europe



XITASO

Coding Agent Harnesses

Why State & Scaffolding Beat the Model

Dr. Simon Stieber - XITASO
appliedAI Developers Meetup · Munich AI Nexus
21 May 2026 | Celonis, Munich

XITASO

Confidential

STRATEGIC SHIFT

Conversation is stuck on the wrong question

“Which AI model is best?“

Performance Impact from Harness

6x

difference on the same underlying model

[SWE-bench Mobile (Feb 2026)]

The Harness Components:

- State management
- Context control
- Tool scaffolding
- Retry logic

This is a larger difference than a recent model generation change.

XITASO

Confidential

The Evidence

Same model. 6x performance gap



XITASO

Confidential

EVIDENCE

Same model. 6× performance gap.

[SWE-bench Mobile (Feb 2026)] — Cursor vs. OpenCode, both on Claude Opus 4.5; 500K-LOC iOS codebase

Cursor

Claude Opus 4.5

12%

task completion

OpenCode

Claude Opus 4.5

2%

task completion

The harness choice > the model choice

The harness choice created a 6x gap on identical infrastructure.

XITASO

Confidential

EVIDENCE

The pattern holds across every source

Benchmark Data

[SWE-bench Mobile (Feb 2026)]

6× gap: Cursor 12% vs. OpenCode 2% (same Opus 4.5)

Sonnet 4.5 ranges 4% - 12% across harnesses

[SWE-bench Verified (ongoing)]

19 pp spread on *Claude 3.5 Sonnet* (33.6% → 53.0%) (June 2024)

Same model, same tasks — only the agent framework differs

[Aider 2023]

35 pp from format choice alone (18% vs. 53%)

Purest experiment: same model, same tasks, single variable changed

XITASO

Confidential

EVIDENCE

The pattern holds

Industry Insights

Cursor engineering blog [Cursor (Apr 2026)]

Shelved expensive model upgrade — "negligible benefit"; harness reliability to 99.9% was the real lever

Anthropic harness study [Anthropic (Mar 2026)]

Solo Opus 4.5: core feature broken. Full harness: functional product, 16 features delivered

Anthropic SWE-bench report [Anthropic (2024)]

"We spent more time optimizing our tools than the overall prompt"

The agentic IDE and model maker prioritizes harness over prompt engineering.

XITASO

Confidential

What's Actually Inside a Harness

Context, checkpointing, recovery, evaluation.

"Plan to invest just as much effort in agent-computer interfaces (ACI) as in human-computer interfaces." [Anthropic (Dec 2024)]

XITASO

Confidential

AGENT ARCHITECTURE OVERVIEW

6 Layers of an Agent

01 Self-Evaluation

02 Checkpointing

03 Retry & Recovery

04 Tool Orchestration

05 Context / Memory → other presentation

06 State Versioning → other presentation

XITASO

Confidential

SELF-EVALUATION & HARNESS INFRASTRUCTURE

Layers of an Agent (II)

> *Self-Evaluation; "Agents confidently praise their own work even when quality is obviously mediocre.“*

Solution: Separate Infrastructure & Claude Code Harness

1. Permission judge (Classifier) — Analyzes conversation intent, auto-approves actions, flags anomalies

2. Content filter (Haiku) — Summarizes web pages before main model sees them

3. Compaction — (Next slide)

The main agent doesn't get a vote. Harness architecture makes the judgement call.
[Anthropic (Mar 2026)]

XITASO

Confidential

CHECKPOINTING & RECOVERY

Layers of an Agent (III)

> *"Persistence for resilience": The harness deciding "I'm about to lose context, let me preserve the important bits."*

Claude Code's 3-Layer Cascading Fallback

Layer 1: MicroCompact (Continuous) — Background token cleanup, trim low-value content

Layer 2: AutoCompact (13K tokens remaining) — Selective compression via relevance scoring, 20K summary budget, circuit breaker

Layer 3: Full Compact (Capacity Crisis) — Emergency: compress everything, re-inject only critical content on 50K budget

The harness decides "I'm about to lose context, let me preserve what matters." Done by a lighter model. Agent itself never knows it happened.

[Huggingface CC architecture (Mar 2026)]

TOOL ORCHESTRATION / EDIT FORMATS

Layers of an Agent (IV)

> *"Wrong format costs additional reasoning tokens and leads to more errors."*
>
> [Cursor (Apr 2026)]

The edit format your harness picks determines how much reasoning budget is left for the actual task.

XITASO

Confidential

Editing modes per model family

What's publicly known about edit format per model

Model family	Best format	Source
Anthropic (Claude)	string-replace	[Cursor (Apr 2026)], Claude Code product design
OpenAI Models of 25/26	patch-based diffs	[Cursor (Apr 2026)]
GPT-3.5 (2023)	whole file	[Aider (2023)]
Gemini, Llama, Mistral, DeepSeek	unknown	No public controlled experiments

XITASO

Confidential

WHAT'S INSIDE A HARNESS — STATE LAYER

Commercial vs. Open-Source

Claude Code / Cursor

Invisible & automatic state management

Context: Auto compaction + dynamic retrieval

Checkpoints: Git-based, auto-rollback

Retry: Built-in harness-level handling

Tools: Typed schemas, per-model formats

State: Transparent continuity

→ **Best benchmark numbers** today

→ Deepest **vendor lock-in**

→ Migrating away is painful

OpenCode / Cline

Explicit, user-managed state primitives

Context: Memory Bank, manual triggers

Checkpoints: Manual save points

Retry: Basic, less graceful degradation

Tools: Configurable MCP

State: Session-based, resets unless exported

→ Portability & **sovereign hosting**

→ More control, more responsibility

→ Lower switching cost

XITASO

Confidential

Open Source Cont’d – Custom Harnesses

Pi (pi.dev) — minimal terminal coding harness

Philosophy: "aggressively extensible so it doesn't dictate your workflow"

15+ providers: subscription or API key

No MCP, sub-agents, or popups *(all buildable via extensions)*

Replace built-in tools entirely; custom compaction & evaluation

Tree-structured sessions with branching and forking

Rich ecosystem: damage-control, cross-agent configs, session replay, and team orchestration

The trade-off: commercial tools embed the development time/testing.
With Pi, that's your homework.



Confidential

Stop arguing about models.
Start arguing about harnesses —
specifically, about state.

XITASO

Confidential

2026

XITASO GmbH

Sources

[SWE-bench Mobile (Feb 2026)] "SWE-bench Mobile: An Evaluation Benchmark for Mobile Software Engineering Agents" ; arxiv.org/abs/2602.09540 ; Feb 2026

[Cursor (Apr 2026)] "Continually Improving Our Agent Harness" ; cursor.com/blog/continually-improving-our-agent-harness ; Apr 30, 2026

[Anthropic (Mar 2026)] "Harness Design for Long-Running Application Development" ; anthropic.com/engineering/harness-design-long-running-apps ; Mar 24, 2026

[Anthropic (2024)] "SWE-bench with Claude" ; anthropic.com/research/swe-bench-sonnet ; 2024

[Aider (2023)] "Aider Edit Format Benchmarks" ; aider.chat/docs/benchmarks ; 2023

[SWE-bench Verified (ongoing)] "SWE-bench Verified Leaderboard" ; swebench.com ; ongoing

[Pi] "Pi Terminal Coding Agent"; pi.dev

[Anthropic (Dec 2024)] "Building Effective Agents"; anthropic.com/engineering/building-effective-agents; Dec 19, 2024

[Huggingface CC architecture (Mar 2026)]; "Production AI Architecture Patterns from 512,000 Lines“; discuss.huggingface.co/t/claude-code-source-leak-production-ai-architecture-patterns-from-512-000-lines/174846; Mar 2026

Questions?

XITASO

Dr. Simon Stieber
Senior AI Engineer

+49 89 69 31 470 110
simon.stieber@xitaso.com

linkedin.com/in/simon-stieber

www.xitaso.com



appliedAI Developers

Challenge Lab

appliedAI
institute
for europe

Make this workflow resumable and debuggable

Workflow:

1. Parse incoming document
2. Extract structured information with an LLM
3. Enrich data via external API
4. Generate final output

Problems:

- LLM outputs vary across runs
- API calls fail intermittently
- retries produce different results
- workflows break mid-execution
- no visibility into intermediate state

appliedAI Developers

Your task

Design a state layer that makes this workflow:

- **resumable after failure**
- **debuggable at every step**
- **reproducible across runs**
- **consistent across workflow versions**

In your group, decide:

1. **What state do you persist?**
2. **Where do you persist it?**
3. **How do you resume safely?**
4. **How do you debug failed runs?**
5. **What tradeoff do you accept?**

Output: one design choice, one tradeoff, one remaining failure mode.

Break

appliedAI Developers

David & Hande

Memory that learns: what agents are still missing

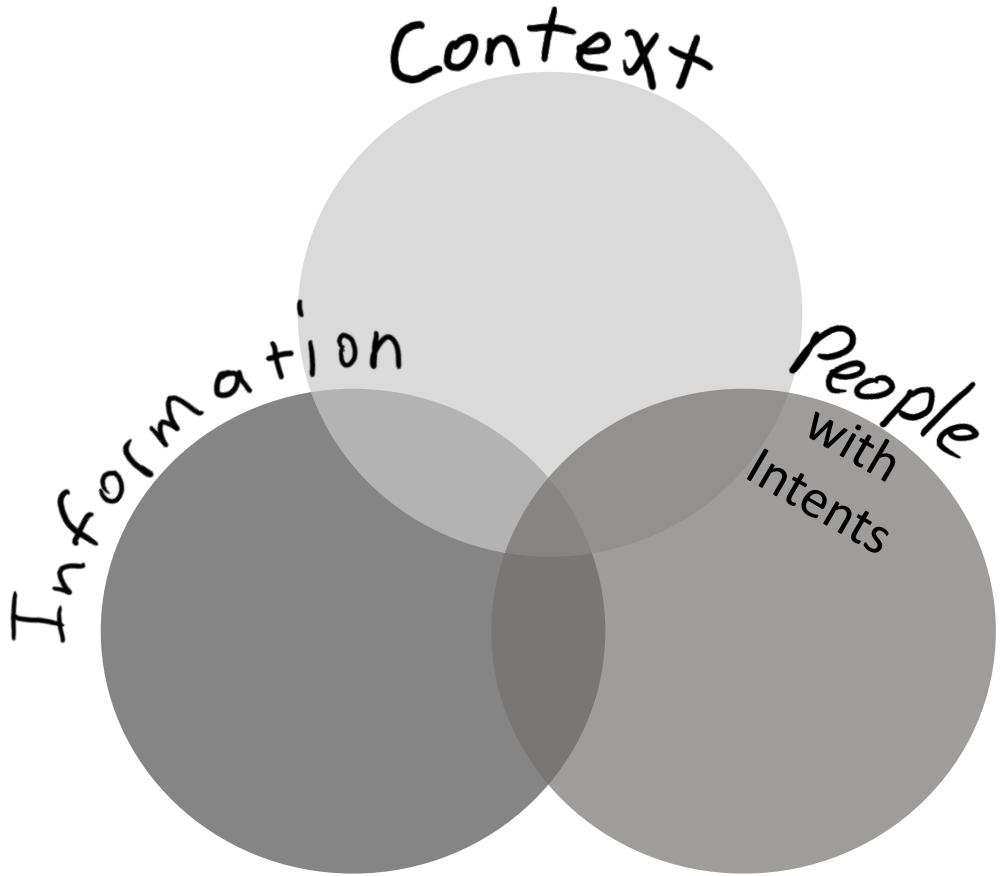
appliedAI institute for europe

“Make it work like Google”

“Make it work like ChatGPT”

“We are agents trying to take action in our environments, and we have to understand those environments well enough to take the appropriate actions.”

Andrew Hinton
Understanding Context: Environment, Language, and Information Architecture, 2014


Context
Information
People
with
Intents

“maximize quality of java beans”

“maximize quality of java beans”



Francesca

- Searching from a café
- Safari
- History of searches about software and platforms

Federico

- Searching from a café
- Chromium
- History of searching about coffees and café operations


JAVA
JAVA
I ♥
COFFEE

“maximize quality of java beans”

Francesca, Java Developer

- Normally in an office, today in a café
- “I am a senior software architect developing applications with the java programming language. Suggest improvements to my code to maximize execution efficiency.”

- Access to documentation, learning materials, code libraries
- **Patterns you expect a developer to follow**
- **What you expect a developer to prioritize**

Context + Intent = Demand Signal
Chunks ≠ Facts
Retrieve facts, generate responses

cognee

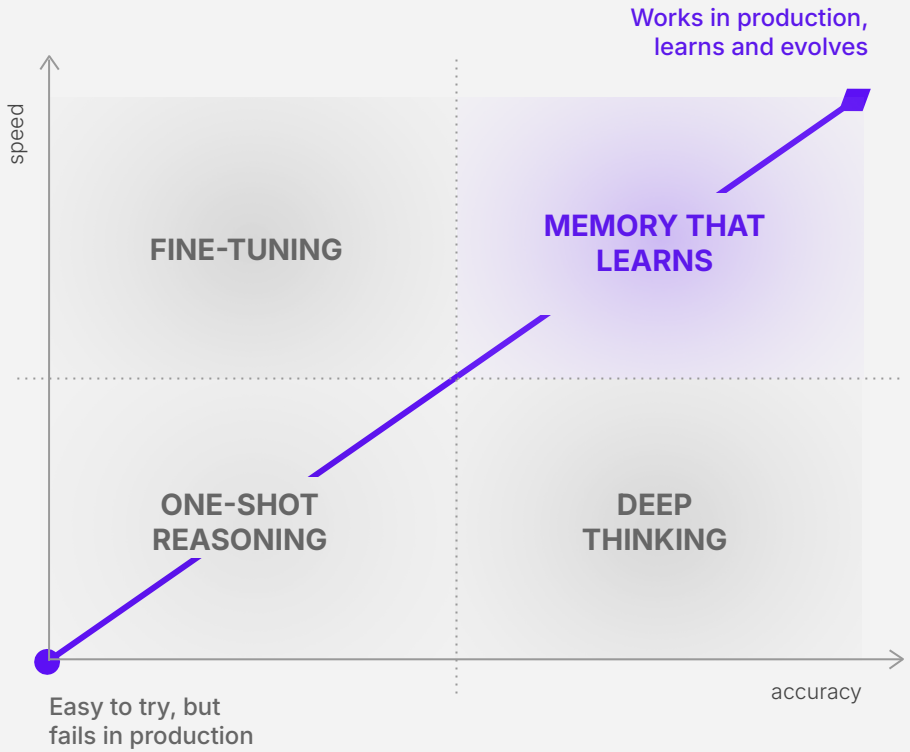
The state of agent context

Most AI journeys start with a single Agent and an MD file.

Seems easy with a few documents, but breaks down as your corpus grows and keeps changing.

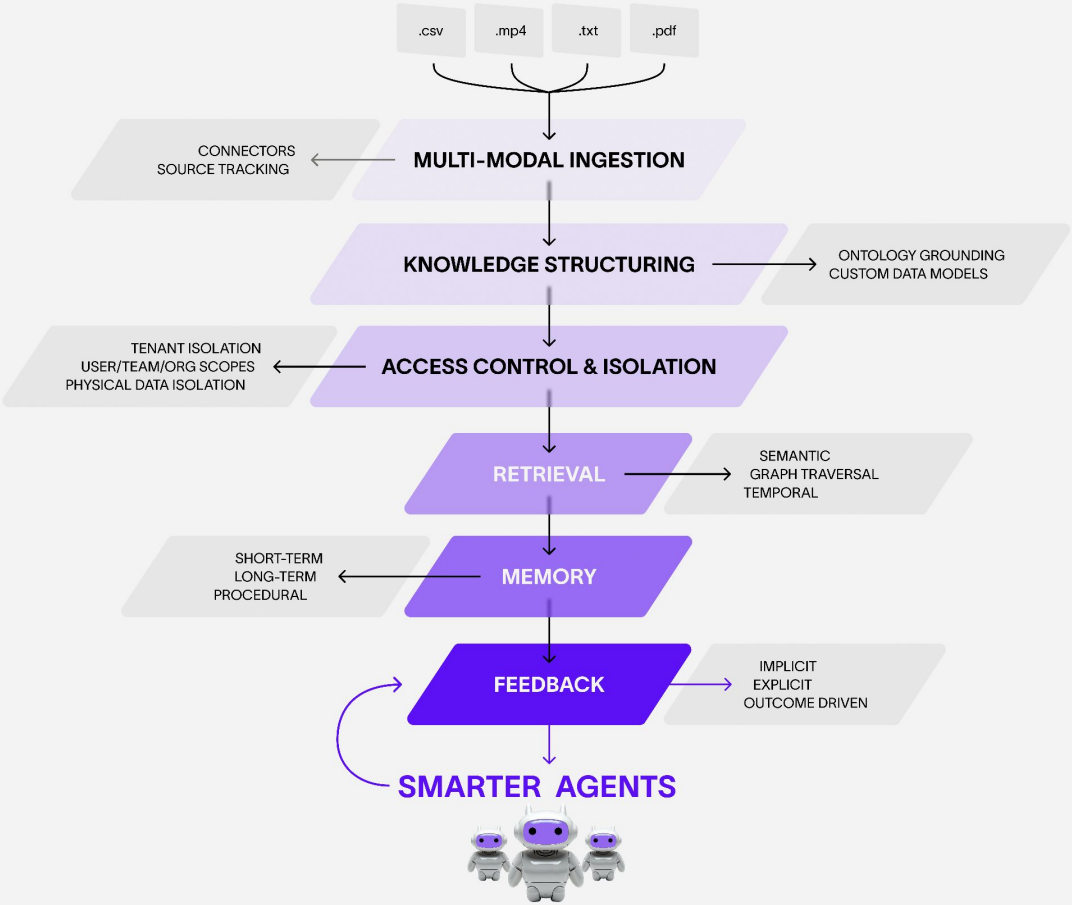
WHAT WE NEED

- Unify context across sources
- Maintain structured, reliable memory
- Reason over relationships
- Learn from outcomes

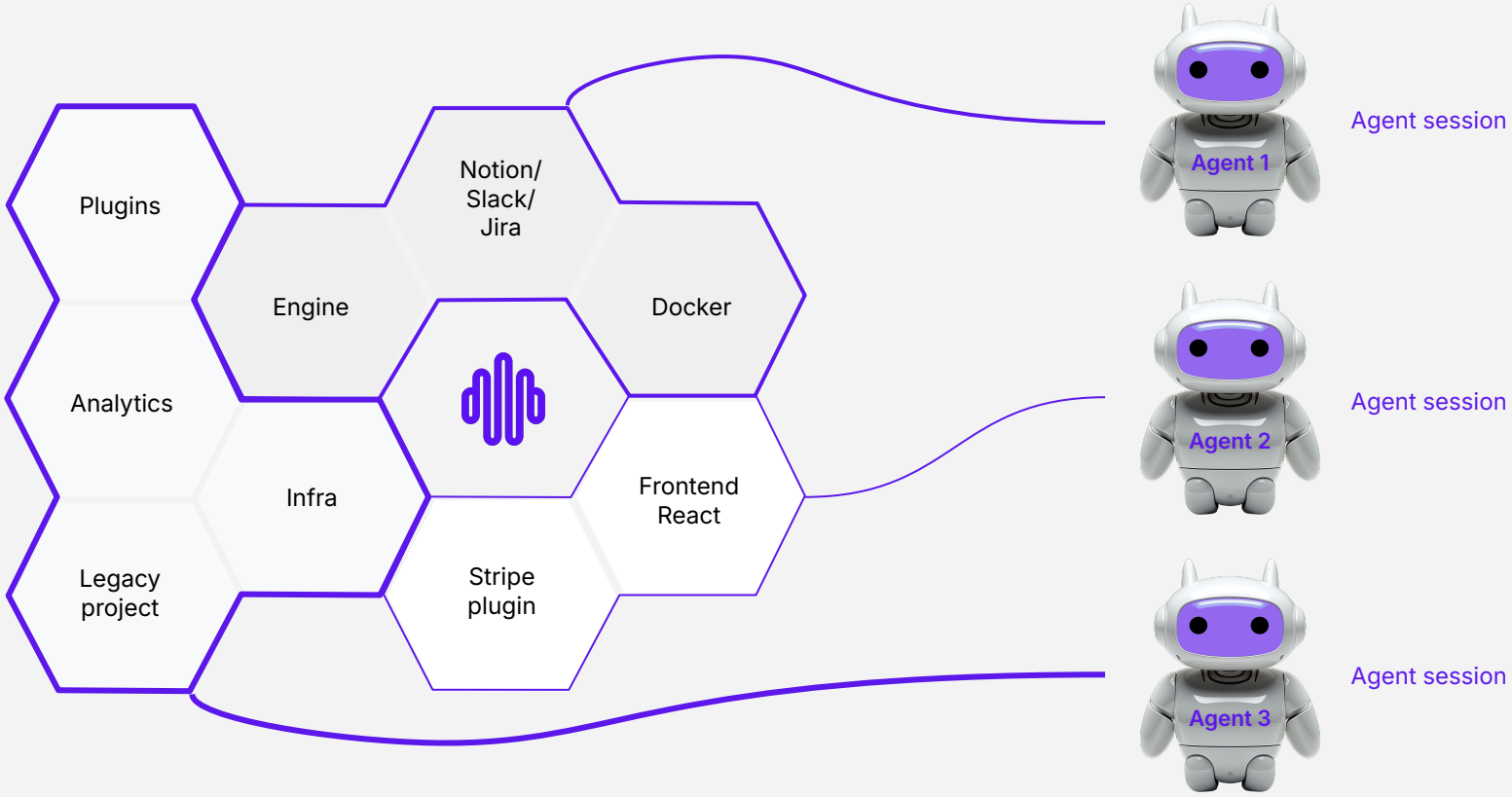


MEMORY ENGINE THAT LEARNS

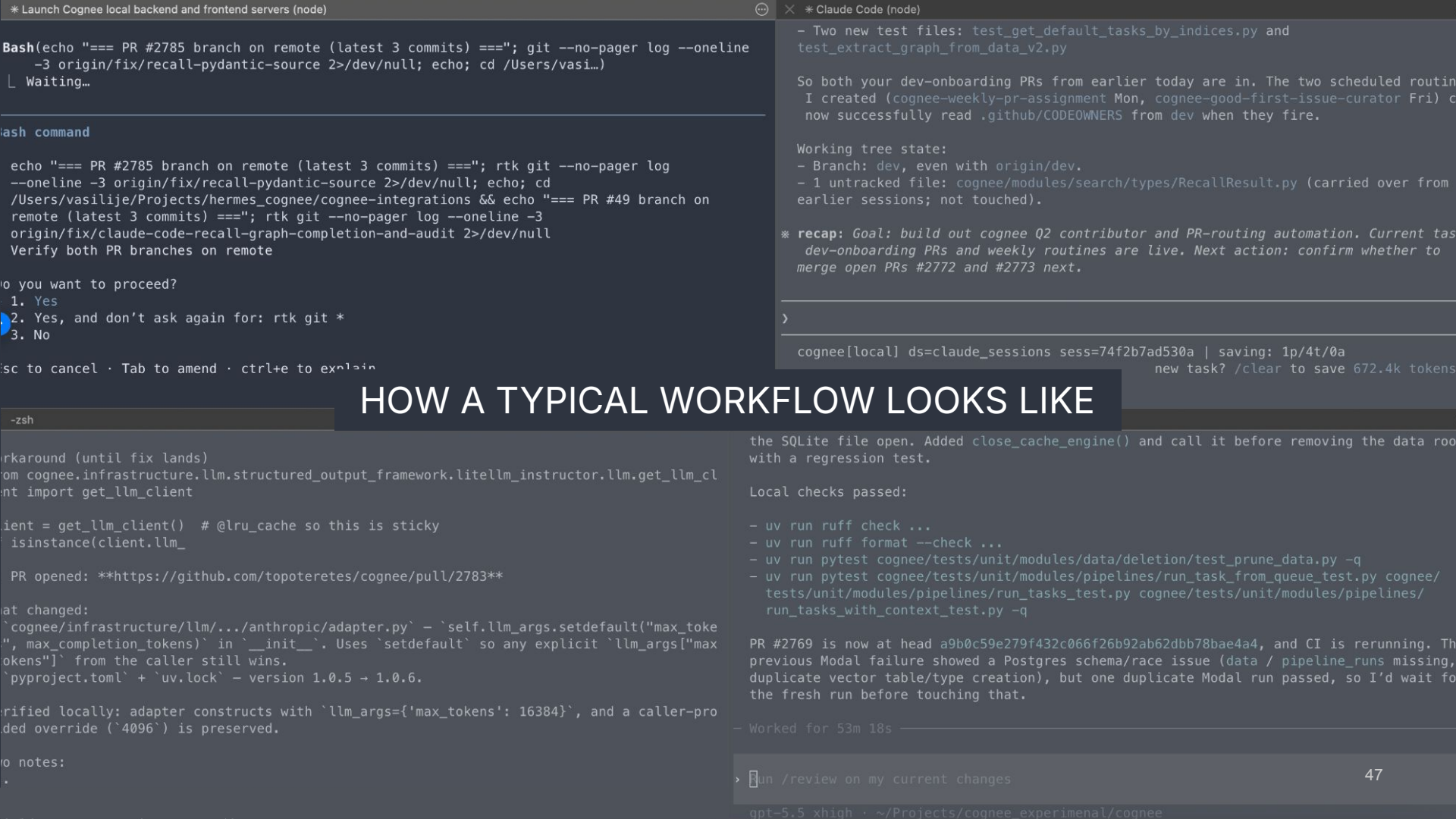
cognee



cognee


Plugins
Notion/
Slack/
Jira
Engine
Docker
Analytics
Infra
Frontend
React
Legacy
project
Stripe
plugin
Agent 1
Agent session
Agent 2
Agent session
Agent 3
Agent session

HOW A TYPICAL WORKFLOW LOOKS LIKE



MEMORY ENGINE THAT LEARNS

cognee



Agent session



Agent session



Agent session

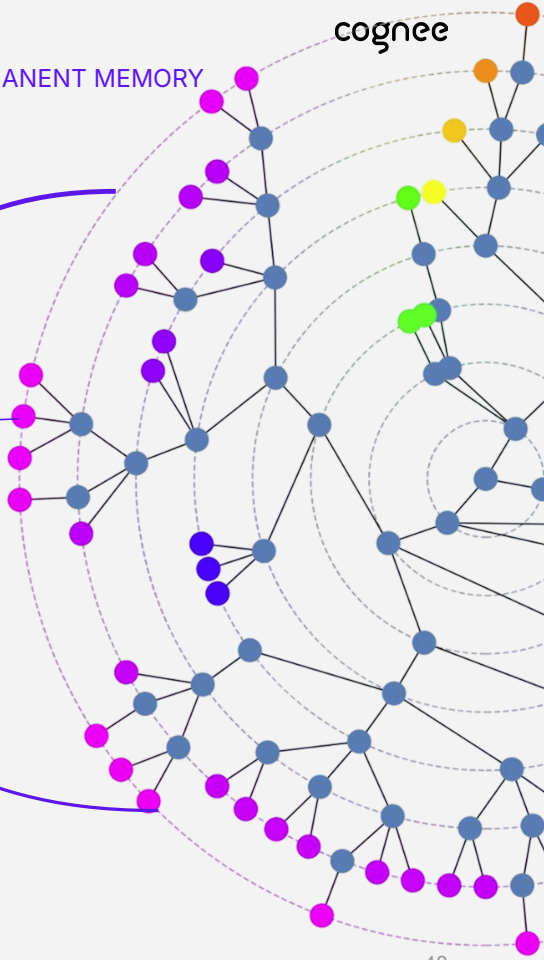
SESSION MEMORY

- QA entries
- trace steps
- feedback

- QA entries
- trace steps
- feedback

- QA entries
- trace steps
- feedback

PERMANENT MEMORY

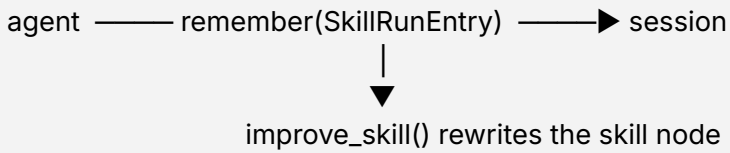


cognee

Pattern 1 — Explicit (typed entries)

```
# Caller composes what to remember. Best for explicit signals.
 await cognee.remember(
    SkillRunEntry(
       selected_skill_id="pr-comment-evaluator",
       success_score=0.30,
       feedback=-1.0,
       result_summary="tone-only judgment missed the bug risk",
    ),
    session_id=SESSION,
 )
 await cognee.improve(session_ids=[SESSION])

# the skill's text in the graph gets rewritten
```



When to reach for it: explicit feedback signals (skill scoring, user thumbs-up, eval-loop verdicts).
You own the structure of what gets saved.

Explore: GitHub

Pattern 2 — Implicit (the @agent_memory decorator)

```
@cognee.agent_memory(
    with_memory=True,                  # recall before
    save_session_traces=True,            # write trace after
    memory_query_from_method="ticket",
    session_id=SESSION,
 )
 async def support_agent(ticket: str, tools) → str:
    # any LLM call inside auto-gets memory prepended to its prompt.
    # status (success/error) is taken from try/except — not hand-written.
    ...
```

wrapped function
    │ before: retrieve_memory_context() ──▶ contextvar
    │ during: LLMGateway prepends memory to every LLM call
    │ after: persist_trace(status from try/except)
    ▼

When to reach for it: you're writing the agent in Python. Adding @agent_memory to entry points and tool functions captures the whole loop with one line per function

Explore: GitHub

cognee

Pattern 3 — Framework hooks (Agent plugins)

```
// hooks.json — Claude Code lifecycle events become cognee writes
 {
  "SessionStart":      "connect, set session_id from cwd",
  "UserPromptSubmit":  "search session+graph, inject as context",
  "PostToolUse":       "append tool name/input/output to session",
  "PreCompact":        "build memory anchor before context reset",
  "SessionEnd":        "cognee.improve() — bridge to permanent graph"
 }
```

Claude Code event ⟶ hook script ⟶ cognee API
(no app code change)

UserPromptSubmit → search & inject
PostToolUse → save trace
SessionEnd → improve()

When to reach for it: the agent runtime isn't yours to modify. Claude Code or any harness with lifecycle hooks. Memory gets captured at the framework boundary without touching the agent's code at all.

Explore: GitHub

cognee

Food for thought

1

If your system grows along any axis, a flat store rebuilds a database.

Users, agents, time, task types. Pick the database you want to build, or one will be built around you by accident.

2

Distillation is one-way. Keep the link to traces — or accept you can't evaluate.

Once a rule has been summarised, the runs that produced it are gone. You've made inference cheaper and evaluation impossible.

3

Memory and skills degrade for the same reason. Build one loop, not two.

Observe, inspect, amend, evaluate. The integration surface (explicit, decorator, hooks) adapts to your code.

appliedAI Developers

Closing Synthesis: State has two jobs

Control

- replay
- resume
- debug
- audit

Learning

- remember useful context
- improve across runs
- incorporate feedback
- reduce repeated failure

State is what turns isolated LLM calls into systems.

appliedAI Developers

Next meetup - July

Collaboration – Human Oversight by Design

Get in touch with us if you want to...

- Host our meetup
- Run a group discussion challenge
- Join as a speaker



Asaad Almutareb
Meetup Moderator &
Co-organiser



Kat Jakubowska
Meetup Initiator &
Organiser

appliedAI
institute
for europe